Prompt caching: prefix reuse, real costs and a measurable demo

Researched 2026-10-12.

What is being cached

The reel's "90% cheaper" headline concerns a repeated input prefix. It does not eliminate output generation, the first write, changed context or failed requests. Current official documentation (https://platform.claude.com/docs/en/build-with-claude/prompt-caching) describes automatic and explicit `cache_control`, five-minute/default and one-hour TTL options, per-model minimums and pricing exceptions.

Correct the simple universal rule

Do not hardcode "all models need 1,024 tokens" or "every read costs 0.1x". The current table has model-specific thresholds and read multipliers. Keep prices as dated configuration in an experiment, and use the exact model/platform table when estimating costs. Cache eligibility does not imply a hit.

Prompt structure

Place stable tools/instructions/reference content before changing user data, and put an explicit breakpoint at the end of the stable prefix where appropriate. Changing early tool definitions or instructions invalidates reuse below them. A timestamp before the breakpoint creates a different prefix each request. Adding a breakpoint is not equivalent to storing an answer.

Reproduction plan

1. Select one supported model and platform; record its current cache minimum and pricing.
2. Use a public long reference document above that minimum and two different short questions.
3. Run an uncached baseline, then a marked first write, then a repeated prefix read. Record the response's cache creation/read and ordinary input/output usage fields.
4. Change one early character and repeat; then restore the original prefix and test again.
5. Repeat after expiry. Calculate total experiment cost including cache writes and output.
6. If testing concurrently, confirm the first write has become available before treating a miss as a failure.

Where this is useful

Repeated document questions, stable tool schemas and sustained conversations can benefit. Short prompts, constantly changing prefixes and long gaps may not. Cache Tax and keepwarm mods concern Claude Code session behavior; API cache controls and subscription billing should not be conflated. A keepwarm process spends requests itself, so measure the net effect.

Proposed output

A small fixture-based simulator can model expiry and prefix changes without any credentials. An optional real API adapter can then verify the predicted usage fields. Export a cost ledger and state exactly which model/date/prices produced it; never label a hypothetical percentage as measured savings.

Reusable project prompt

```
Build Cache ledger experiment. Prefix/TTL simulator with an optional documented API adapter and total-cost ledger. Read the linked official docs first. Use fixture data; keep credentials outside source control. Deliver tests, setup instructions, an evidence/limitations note and a small reproducible example. Do not claim a measurement you did not run. Milestones: Model prefix hashes and TTL; Create write/read/changed-prefix cases; Compare against real usage fields; Export dated total-cost report.
```

Delivery and next step

The linked current sources were retrieved for this guide; the original dossier and its extraction caveats are retained in Sources. The guide and printable PDF are research deliverables. The proposed project has not been built. Start with the first milestone and verify its acceptance criteria before expanding the scope.

Original sources

- https://www.instagram.com/reel/DdwqtoNIGus/